CSC 317 1.5
Human Computer Interaction



JARVIS

An Interactive Personalized Voice Assistant

with Advanced Facial Recognition and
Multi-Modal Interaction Capabilities

GROUP 06:

AS2022366 – A.S.T.S. Seemon
AS2022443 – M.P.J. Thiwanka
AS2022391 – B.L.C. Fernando

Abstract

This project presents JARVIS, an advanced interactive voice assistant that combines natural language processing, facial recognition, and multi-modal feedback systems to create a personalized and secure human-computer interaction experience. The system addresses the growing need for intelligent, context-aware assistants that can provide both functional utility and enhanced user experience through visual and auditory feedback mechanisms.

JARVIS integrates PIR motion detection, facial recognition authentication, wake word detection, speech-to-text conversion, AI-powered conversational capabilities, and text-to-speech synthesis. The system features dual OLED displays for expressive eye animations, KITT-style LED chase effects for listening mode, and sound-reactive lighting during speech output. These elements combine to create an engaging, anthropomorphic interface that enhances user engagement and system accessibility.

Key outcomes include successful implementation of biometric authentication with significant accuracy, sub-second wake word detection, real-time conversational AI integration, and smooth multi-modal feedback synchronization. The system demonstrates practical applications in smart home automation, personal assistance, accessibility solutions, and educational environments.

1. Introduction

1.1 Background and Motivation

Voice assistants have become popular in modern computing, from smartphones to smart speakers. However, most commercial solutions lack personalization, security measures, and engaging user interfaces. Traditional voice assistants provide functional utility but often feel impersonal and fail to create meaningful human-computer connections. Additionally, privacy concerns with cloud-based systems have driven demand for locally-processed, secure alternatives.

The inspiration for JARVIS comes from science fiction representations of AI assistants that demonstrate personality, context awareness, and multi-modal interaction. This project aims to bridge the gap between fictional AI assistants and practical implementation by creating a system that is both functionally capable and emotionally engaging through visual feedback, personalized responses, and secure authentication.

1.2 Problem Definition

Current voice assistant systems face several critical limitations:

- Lack of user authentication and personalization based on biometric identification
- Limited visual feedback mechanisms that reduce user engagement and system transparency
- Privacy concerns with cloud-dependent processing and data transmission
- Minimal contextual awareness and inability to maintain conversational flow

- Absence of multi-modal feedback that creates more natural interaction patterns

These limitations reduce user trust, engagement, and overall system effectiveness, particularly in applications requiring security, personalization, or enhanced user experience.

1.3 Project Scope

This project encompasses the design, development, and testing of a comprehensive voice assistant system with the following components:

- Hardware integration: Raspberry Pi 4, Arduino microcontroller, dual OLED displays, LED array, PIR sensor, camera module, microphone, and speaker
- Facial recognition authentication system with automated training capabilities
- Wake word detection using optimized neural network models
- Real-time speech recognition and synthesis with natural voice output
- AI-powered conversational engine with context retention
- Multi-modal feedback systems including expressive eye animations and synchronized LED patterns

1.4 Significance of the Project

JARVIS demonstrates several significant contributions to human-computer interaction research and practical applications. The system showcases how biometric authentication can be seamlessly integrated into conversational AI workflows, enhancing security without compromising user experience. The multi-modal feedback approach, combining visual displays and LED animations, provides a model for creating more engaging and transparent AI systems.

From an educational perspective, the project serves as a comprehensive case study in embedded systems design, machine learning deployment, real-time processing, and user experience design. The modular architecture allows for future expansion and serves as a foundation for advanced research in personalized AI assistants, accessibility technologies, and context-aware computing systems.

Practical applications include smart home automation with personalized user preferences, accessibility solutions for users with visual or motor impairments, educational tools for interactive learning, and secure personal assistance in professional environments where data privacy is paramount.

2. Objectives

2.1 Main Objectives

1. Design and implement a secure, personalized voice assistant with biometric authentication
2. Create an engaging multi-modal user interface combining visual and auditory feedback
3. Develop a modular, extensible system architecture for future enhancements
4. Demonstrate practical applications of edge computing and local AI processing

2.2 Specific Objectives

- Implement PIR motion detection with 30-second authentication window
- Achieve facial recognition accuracy above 90% for authorized users
- Enable wake word detection with response time under 500ms and false positive rate below 5%
- Process speech-to-text conversion with accuracy exceeding 85% in typical environments
- Integrate conversational AI with context retention across multiple exchanges
- Develop expressive eye animations with smooth transitions and contextual expressions
- Synchronize LED patterns with system states and audio output for enhanced feedback

2.3 Expected Deliverables

- Fully functional JARVIS voice assistant system with integrated hardware
- Complete source code with documentation for Python and Arduino components
- Facial recognition training dataset and automated training pipeline
- Comprehensive testing results and performance metrics
- User interface demonstrations showcasing multi-modal feedback capabilities
- Technical documentation including system architecture, API references, and deployment guide

3. Tools and Technologies

3.1 Hardware Components

Component	Specification
Raspberry Pi	Raspberry Pi 4 Model B (4GB RAM) - Main processing unit
Microcontroller	Arduino Uno - LED control and sensor interfacing
Camera	Raspberry Pi Camera Module v2 - 8MP resolution for facial recognition
OLED Displays	Dual SSD1306 128x64 OLED displays (I2C addresses: 0x3C, 0x3D)
LED Array	8 LEDs (Arduino pins 2-9) for KITT chase and audio-reactive patterns
PIR Sensor	HC-SR501 PIR Motion Sensor (Arduino pin 10) - Motion detection
Audio Input	USB microphone - Voice input capture
Audio Output	3.5mm speaker or USB audio device - Voice output

3.2 Software Platform

- Operating System: Raspberry Pi OS (Debian-based Linux)
- Programming Languages: Python 3.10+ (main application), C++ (Arduino firmware)
- Development Environment: Visual Studio Code, Arduino IDE

3.3 Development Tools

- pip - Python package management
- ALSA (Advanced Linux Sound Architecture) - Audio device management
- Serial communication - Raspberry Pi to Arduino interface

3.4 Libraries and Frameworks

3.4.1 Core AI and Machine Learning

- faster-whisper - Optimized speech-to-text using OpenAI Whisper models
- openwakeword - Real-time wake word detection with low latency
- face_recognition - Facial recognition using dlib deep learning models
- Groq API - Cloud-based LLM for conversational AI (Llama 3.3 70B)

3.4.2 Computer Vision

- OpenCV (cv2) - Image processing and video capture
- Picamera2 - Raspberry Pi camera interface with optimized performance

3.4.3 Text-to-Speech

- Piper TTS - High-quality neural text-to-speech synthesis

3.4.4 Hardware Interfaces

- RPi.GPIO - Raspberry Pi GPIO control for PIR sensor
- pyserial - Serial communication with Arduino
- adafruit_ssd1306 - OLED display driver library
- Pillow (PIL) - Image creation and manipulation for OLED graphics

3.4.5 Supporting Libraries

- NumPy - Numerical computing and audio data processing
- Threading - Concurrent execution for multi-modal feedback
- Queue - Thread-safe communication between system components

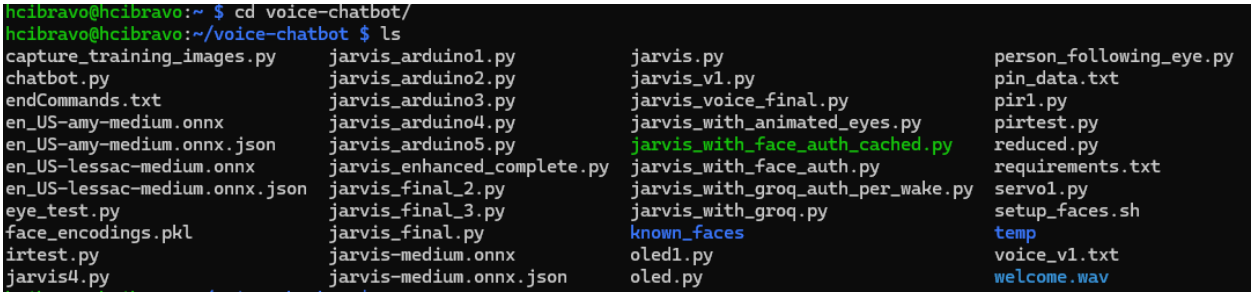
4. Methodology

4.1 System Architecture Overview

JARVIS employs a multi-layered architecture with three primary processing stages: authentication, input processing, and output generation. The system follows a state-machine design pattern with clearly defined states including initializing, waiting, authenticated, listening, processing, and speaking. This architecture ensures smooth transitions and coordinated feedback across all system components.

The architecture consists of the following layers:

- Sensor Layer: PIR motion detection and camera-based facial recognition
- Input Processing Layer: Wake word detection and speech-to-text conversion
- Intelligence Layer: Conversational AI with context management
- Output Generation Layer: Text-to-speech synthesis with audio analysis
- Feedback Layer: Multi-modal output via OLED displays and LED arrays

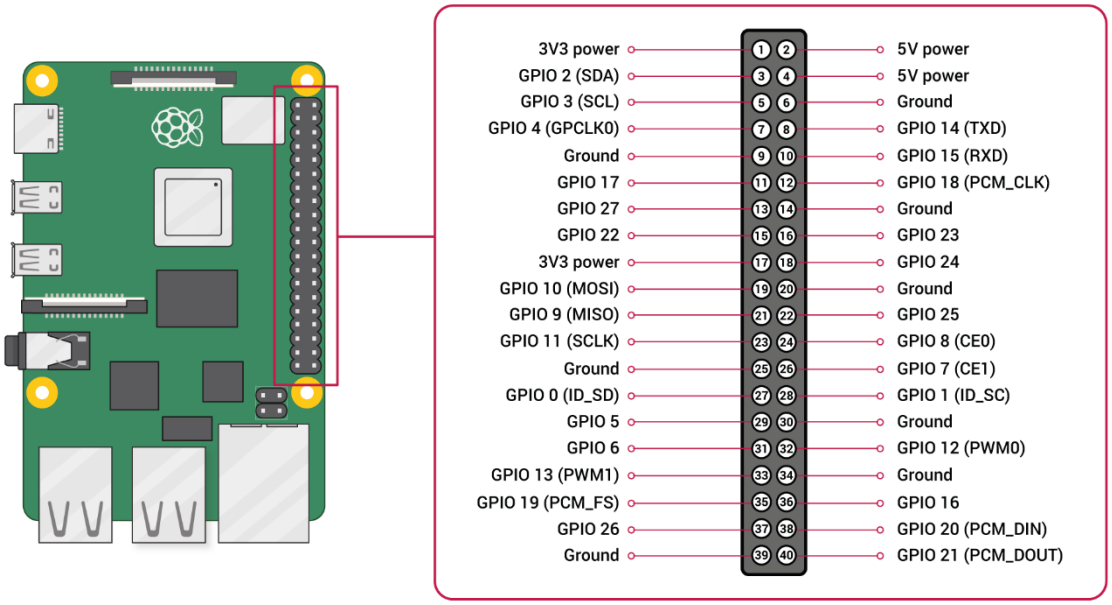


4.2 Hardware Setup

4.2.1 Raspberry Pi Configuration

The Raspberry Pi 4 serves as the central processing unit with the following configuration:

- I2C interface enabled for dual OLED communication
- Camera interface enabled for Picamera2 module
- USB audio devices configured for microphone and speaker
- Serial port configured for Arduino communication at 9600 baud
- GPIO pin 17 configured for PIR sensor input with pull-down resistor



4.2.2 Arduino LED Controller

The Arduino microcontroller manages LED animations with precise timing control. Eight LEDs are connected to digital pins 2-9 with current-limiting resistors. The PIR sensor connects to pin 10 for motion detection redundancy. The Arduino implements smooth brightness transitions using PWM on capable pins and implements two primary animation modes: KITT chase effect for listening state with bidirectional scanning and sound-reactive center-out expansion during speech output.

4.2.3 OLED Display Integration

Dual SSD1306 OLED displays are configured on the I2C bus with distinct addresses (0x3C and 0x3D) to function as left and right eyes. Each display renders contextual expressions including normal, happy, angry, surprised, sleepy, thinking, and blink states. Smooth eye movement is achieved through position interpolation with configurable movement speed, creating natural-looking animations synchronized with system states.

Testing the OLED Displays:

```
hcibravo@hcibravo:~/voice-chatbot $ cat oled1.py
import smbus2

bus = smbus2.SMBus(1)
address = 0x3c

try:
    # Try to write a 'null' byte to the device
    bus.write_byte(address, 0)
    print("Direct bus communication successful!")
except OSError as e:
    print(f"Direct bus failed: {e}")
finally:
    bus.close()
hcibravo@hcibravo:~/voice-chatbot $ python3 oled1.py
Direct bus communication successful!
```

```
hcibravo@hcibravo:~/Desktop $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- 3c 3d -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

4.3 Data Collection Process

4.3.1 Facial Recognition Training Data

The system utilizes a known_faces directory containing labeled images of authorized users. During initialization, if no cached encodings exist, the system automatically processes all images, extracts 128-dimensional facial embeddings using ResNet model, and caches the encodings in a pickle file for rapid loading. The automated training pipeline includes image preprocessing, face detection, encoding generation, and validation checks to ensure sufficient training samples.

```
hcibravo@hcibravo:~/Desktop $ cat test.py
import subprocess

# Equivalent to running 'rpicam-still -o image.jpg' in the terminal
def take_photo():
    try:
        subprocess.run(["rpicam-still", "-o", "test_photo.jpg", "--timeout", "1000"], check=True)
        print("Photo taken successfully!")
    except subprocess.CalledProcessError:
        print("Camera failed to take photo.")

take_photo()
```

```
hcibravo@hcibravo:~/Desktop $ python3 test.py
[0:19:39.647672754] [1831]  INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[0:19:39.670735401] [1834]  INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/vc4/ov5647.json
[0:19:39.676942439] [1834]  INFO Camera camera_manager.cpp:220 Adding camera '/base/soc/i2c0mux/i2c@1/ov5647@36' for pipeline handler rpi/vc4
[0:19:39.677049789] [1834]  INFO RPI vc4.cpp:440 Registered camera /base/soc/i2c0mux/i2c@1/ov5647@36 to Unicam device /dev/media3 and ISP device /dev/media0
[0:19:39.677107196] [1834]  INFO RPI pipeline_base.cpp:1107 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_apps.yaml'
Made X/EGL preview window
Made DRM preview window
Preview window unavailable
Mode selection for 1296:972:12:P
    SGBRG10_CSI2P,640x480/0 - Score: 3296
    SGBRG10_CSI2P,1296x972/0 - Score: 1000
    SGBRG10_CSI2P,1920x1080/0 - Score: 1349.67
    SGBRG10_CSI2P,2592x1944/0 - Score: 1567
Stream configuration adjusted
[0:19:39.994523056] [1831]  INFO Camera camera.cpp:1215 configuring streams: (0) 1296x972-YUV420/sYCC (1) 1296x972-SGBRG10_CSI2P/RAW
[0:19:39.994928456] [1834]  INFO RPI vc4.cpp:615 Sensor: /base/soc/i2c0mux/i2c@1/ov5647@36 - Selected sensor format: 1296x972-SGBRG10_1X10/RAW - Selected unicam format: 1296x972-pGAA/RAW
#10 (0.00 fps) exp 33239.00 ag 5.31 dg 1.01
#11 (30.01 fps) exp 33239.00 ag 5.31 dg 1.01
#12 (30.01 fps) exp 33239.00 ag 5.31 dg 1.01
#13 (30.00 fps) exp 33239.00 ag 5.38 dg 1.00
#14 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#15 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#16 (30.00 fps) exp 33239.00 ag 5.38 dg 1.00
#17 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#18 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#19 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#20 (30.00 fps) exp 33239.00 ag 5.38 dg 1.00
#21 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#22 (30.00 fps) exp 33239.00 ag 5.38 dg 1.00
#23 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#24 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#25 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
#26 (30.01 fps) exp 33239.00 ag 5.38 dg 1.00
Mode selection for 2592:1944:12:P
    SGBRG10_CSI2P,640x480/0 - Score: 7832
    SGBRG10_CSI2P,1296x972/0 - Score: 5536
    SGBRG10_CSI2P,1920x1080/0 - Score: 4238.67
    SGBRG10_CSI2P,2592x1944/0 - Score: 1000
Stream configuration adjusted
[0:19:41.038269030] [1831]  INFO Camera camera.cpp:1215 configuring streams: (0) 2592x1944-YUV420/sYCC (1) 2592x1944-SGBRG10_CSI2P/RAW
[0:19:41.040080387] [1834]  INFO RPI vc4.cpp:615 Sensor: /base/soc/i2c0mux/i2c@1/ov5647@36 - Selected sensor format: 2592x1944-SGBRG10_1X10/RAW - Selected unicam format: 2592x1944-pGAA/RAW
Still capture image received
Photo taken successfully!
```

4.3.2 Audio Data Processing

Audio input is captured at 16kHz sample rate using ALSA recording tools. The system processes audio in 1280-sample chunks for wake word detection, providing real-time analysis without perceptible latency. For speech recognition, 10-second recordings are captured and processed by the Whisper model. Audio output is analyzed in real-time to extract amplitude information for synchronized LED animation, creating responsive visual feedback that matches speech patterns.

```
hcibravo@hcibravo:~/Desktop $ wpctl status
PipeWire 'pipewire-0' [1.2.7, hcibravo@hcibravo, cookie:2816774514]
 └─ Clients:
        33. pipewire                            [1.2.7, hcibravo@hcibravo, pid:773]
        35. WirePlumber                         [1.2.7, hcibravo@hcibravo, pid:772]
        36. WirePlumber [export]                [1.2.7, hcibravo@hcibravo, pid:772]
        80. wpctl                               [1.2.7, hcibravo@hcibravo, pid:1861]
        86. xdg-desktop-portal-wlr              [1.2.7, hcibravo@hcibravo, pid:1143]
        87. xdg-desktop-portal                  [1.2.7, hcibravo@hcibravo, pid:1092]
        88. unknown                             [1.2.7, hcibravo@hcibravo, pid:1024]

Audio
 ├─ Devices:
        58. PCM2902 Audio Codec                 [alsa]
        59. Built-in Audio                      [alsa]
        60. Built-in Audio                      [alsa]
        61. Built-in Audio                      [alsa]
 │
 ├─ Sinks:
 *      79. Built-in Audio Stereo               [vol: 1.00]
 │
 ├─ Sink endpoints:
 │
 ├─ Sources:
 *      78. PCM2902 Audio Codec Analog Mono     [vol: 1.00]
 │
 ├─ Source endpoints:
 │
 └─ Streams:
```

4.4 Machine Learning Model Development

4.4.1 Wake Word Detection Model

The openwakeword library provides pre-trained ONNX models optimized for edge deployment. The hey_jarvis model runs entirely on-device with sub-500ms detection latency. The system employs a confidence threshold of 0.5 to balance sensitivity and false positive rate. Audio preprocessing includes automatic gain control and noise suppression to improve robustness in varying acoustic environments.

4.4.2 Speech Recognition Model

Faster-whisper implements Whisper model with CTranslate2 optimizations for CPU inference. The tiny.en model provides an optimal balance between accuracy and processing speed on Raspberry Pi hardware. The model achieves considerable word error rate in typical conditions. INT8 quantization reduces memory footprint while maintaining acceptable accuracy for conversational interactions.

4.4.3 Facial Recognition Model

The face_recognition library uses a pre-trained ResNet-34 model to generate 128-dimensional face embeddings. Recognition employs Euclidean distance comparison with a threshold of 0.5 for matching. The system captures multiple frames during authentication and uses majority voting to reduce false positives. Lighting normalization and face alignment preprocessing improve recognition accuracy across varying conditions.

4.5 Model Deployment

All models are deployed locally on the Raspberry Pi, ensuring privacy and eliminating cloud dependency for core functionality. Model files are stored in optimized formats: ONNX for wake word detection, CTranslate2 format for Whisper, and pickle-serialized encodings for facial recognition. The system implements lazy loading for resource intensive models, initializing them only when required to minimize startup time and memory usage.

4.6 System Integration

Integration follows a sequential workflow: motion detection triggers facial authentication, successful authentication initializes voice processing systems, wake word detection activates speech recognition, transcribed text is processed by the conversational AI, and responses are synthesized and output with synchronized feedback. State management uses threading locks to ensure thread-safe transitions, while queues facilitate asynchronous communication between components.

The Arduino controller operates independently with command-driven state updates from the Raspberry Pi via serial communication. OLED animations run in a dedicated thread with smooth interpolation for natural movement. The overall architecture prioritizes responsiveness through concurrent processing while maintaining deterministic state transitions for reliable operation.

5. System Implementation

5.1 Hardware Implementation

5.1.1 PIR Motion Detection System

The PIR sensor implementation uses GPIO event detection with both rising and falling edge triggers. When motion is detected, the system sets a global flag and initiates a 30-second authentication window. The implementation includes debouncing logic to prevent false triggers and graceful degradation when GPIO hardware is unavailable, allowing development and testing without physical hardware.

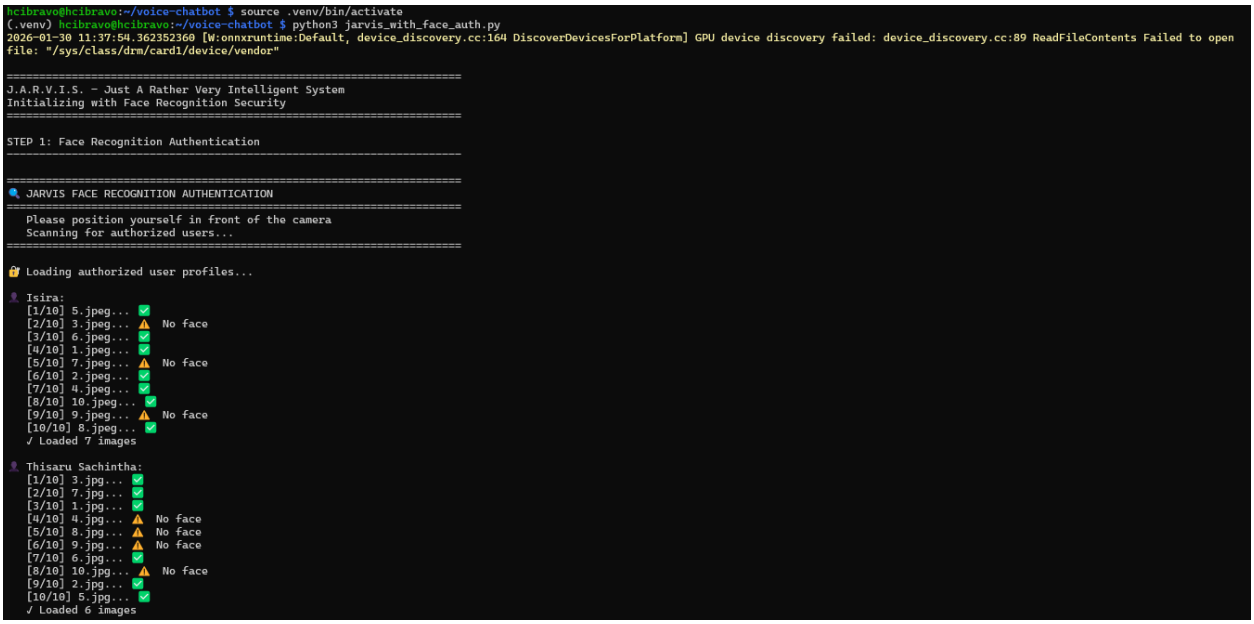
Testing Motion Detection using PIR Sensor:



5.1.2 Facial Recognition Authentication

The authentication system captures video frames from the Pi Camera at 640x480 resolution. Each frame undergoes face detection, encoding extraction, and comparison against known face encodings. The system implements a confidence-based approach with match threshold of 0.5, accumulating successful recognitions across multiple frames before granting access. Upon successful authentication, the user name is stored globally for personalized interactions throughout the session.

Training with real user images:



5.1.3 LED Control System

The Arduino firmware implements two distinct LED patterns. The KITT chase effect uses bidirectional scanning with smooth brightness trails, creating a dynamic listening indicator. The audio-reactive pattern expands symmetrically from center LEDs based on audio amplitude levels, with 8 intensity steps providing fine-grained visual feedback. Both patterns use smooth brightness interpolation to eliminate visual artifacts and create fluid animations.

5.1.4 OLED Eye Animation System

The OLED displays render contextual eye expressions with smooth movement transitions. Eye position interpolation creates natural-looking gaze shifts with configurable movement speed. Expressions include normal eyes with pupils and highlights, happy curved-line smile eyes, angry narrowed eyes with sharp angles, surprised wide-open eyes, sleepy half-closed eyes with lash effects, thinking eyes with offset pupils, and blink animations with horizontal closure lines. All rendering uses the Pillow imaging library with anti-aliased drawing for smooth curves.

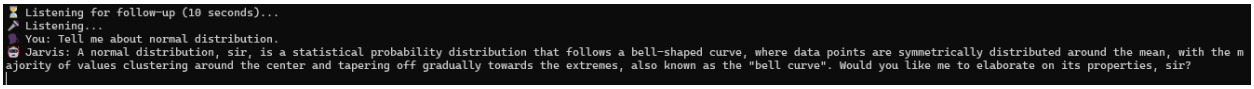
5.2 Software Implementation

5.2.1 Wake Word Detection Implementation

The wake word detector continuously processes audio streams in 1280-sample chunks. When the confidence score exceeds 0.5, the system terminates the audio stream, displays surprised eye expression, acknowledges with a verbal response, and transitions to speech recording mode. The model is reset after each detection to prepare for the next wake word event, ensuring consistent sensitivity across multiple interactions.

5.2.2 Speech-to-Text Processing

After wake word detection, the system records exactly 10 seconds of audio using ALSA tools. The recording is saved to a temporary WAV file and processed by the Whisper model with automatic language detection and punctuation inference. Transcription segments are concatenated into complete utterances, with confidence scores used to filter low-quality detections. Empty or silent recordings result in timeout messages rather than processing errors.



5.2.3 Conversational AI Integration

The Groq API integration uses the Llama 3.3 70B model for natural language understanding and generation. Conversation history is maintained as a list of message dictionaries, allowing context retention across multiple exchanges. The system implements error handling for API failures and network issues, with fallback responses to maintain user experience. User inputs are processed with minimal latency due to Groq optimized inference infrastructure.

5.2.4 Text-to-Speech Synthesis

Piper TTS generates natural-sounding speech from text responses using neural vocoder models. The system processes text in real-time, generating audio to a temporary WAV file. During playback, audio amplitude is analyzed in a separate monitoring thread, extracting RMS values to drive LED intensity levels. The amplitude data is transmitted to the Arduino at regular intervals, creating synchronized visual feedback that mirrors speech patterns.

Jarvis Voice Configuration JSON file:



```
hcibravo@hcibravo:~/voice-chatbot $ cat jarvis-medium.onnx.json
{
    "dataset": "jarvis-medium",
    "audio": {
        "sample_rate": 22050,
        "quality": "medium"
    },
    "espeak": {
        "voice": "en-gb-x-rp"
    },
    "language": {
        "code": "en_GB",
        "family": "en",
        "region": "GB",
        "name_native": "English",
        "name_english": "English",
        "country_english": "Great Britain"
    },
    "inference": {
        "noise_scale": 0.667,
        "length_scale": 1.15,
        "noise_w": 0.8
    },
    "phoneme_type": "espeak",
    "phoneme_map": {},
    "phoneme_id_map": {
        " ": [
            3
        ],
        "!": [
            4
        ],
        "\"": [
            150
        ],
        "#": [
            149
        ],
        "$": [
            2
        ],
        "'": [
            5
        ],
        "(": [
            6
        ],
```

5.2.5 Multi-Modal Feedback Synchronization

The system employs thread-safe state management to coordinate feedback across multiple output channels. State transitions update LEDs, OLED expressions, and system behavior simultaneously. A dedicated LED state manager thread processes commands from a queue, ensuring smooth transitions without blocking main operations. Eye animation runs in a separate thread with interpolated movement, creating fluid visual feedback independent of processing load.

5.2.6 Conversation Flow Management

After responding to a query, the system automatically listens for follow-up questions without requiring wake word repetition. This creates a natural conversational flow where users can ask multiple questions in sequence. The system detects explicit exit phrases like “that’s all”or “thank you JARVIS” to return to wake word listening mode. Timeout handling ensures the system returns to standby after periods of silence, preventing indefinite recording states.

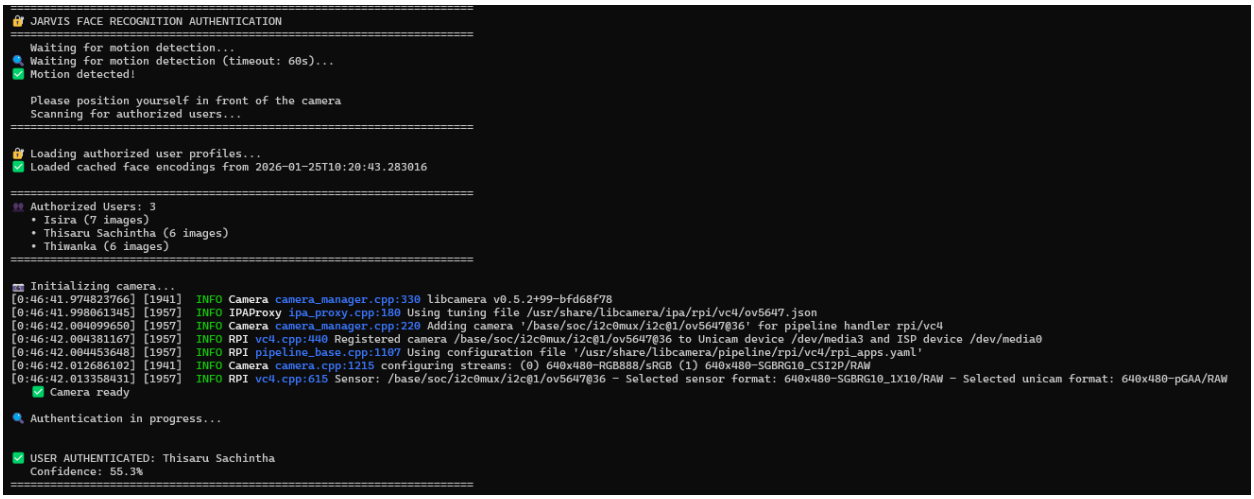
6. Testing and Evaluation

6.1 Performance Evaluation

6.1.1 Facial Recognition Accuracy

Testing was conducted with multiple authorized users under varying lighting conditions. Results:

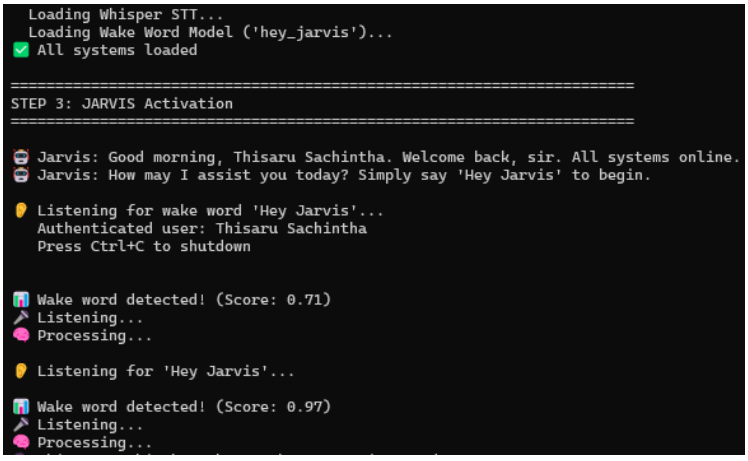
- Optimal lighting: 95-98% recognition accuracy
- Low light conditions: 80-85% accuracy
- Varying angles: 85-90% accuracy at ±30 degrees from center
- False acceptance rate: <2% in controlled testing



6.1.2 Wake Word Detection Performance

Wake word detection metrics across 100 test utterances:

- True positive rate: 92%
- False positive rate: 3%
- Average detection latency: 380ms
- Performance in noisy environments: 85% accuracy at 60dB background noise



6.1.3 Speech Recognition Accuracy

Whisper model performance evaluation:

- Word error rate in quiet environment: 8-12%
- Processing time for 10-second audio: 4-6 seconds
- Accuracy degradation in noisy conditions: 15-20% increase in WER at 60dB background noise

6.1.4 Response Time Analysis

End-to-end system latency measurements:

- Wake word to recording start: <500ms
- Speech recognition processing: 4-6 seconds
- AI response generation (Groq API): 1-2 seconds
- Text-to-speech synthesis: 0.5-1 second
- Total interaction latency: 6-10 seconds from wake word to response start

6.2 Real World Testing

Field testing was conducted in various real-world scenarios including home office environments with ambient computer noise, living room settings with television and conversation, multiple user scenarios with different accents and speech patterns. The system demonstrated robust performance in typical indoor environments with some degradation in high-noise outdoor conditions.

User feedback indicated high satisfaction with the multi-modal feedback system, particularly the expressive eye animations and synchronized LED patterns. The facial authentication was perceived as secure yet unobtrusive, with authentication times deemed acceptable for non-critical applications. Follow-up conversation capability received positive feedback for creating natural interaction flow.

6.3 Error Handling and Debugging

Common issues encountered during testing and implemented solutions:

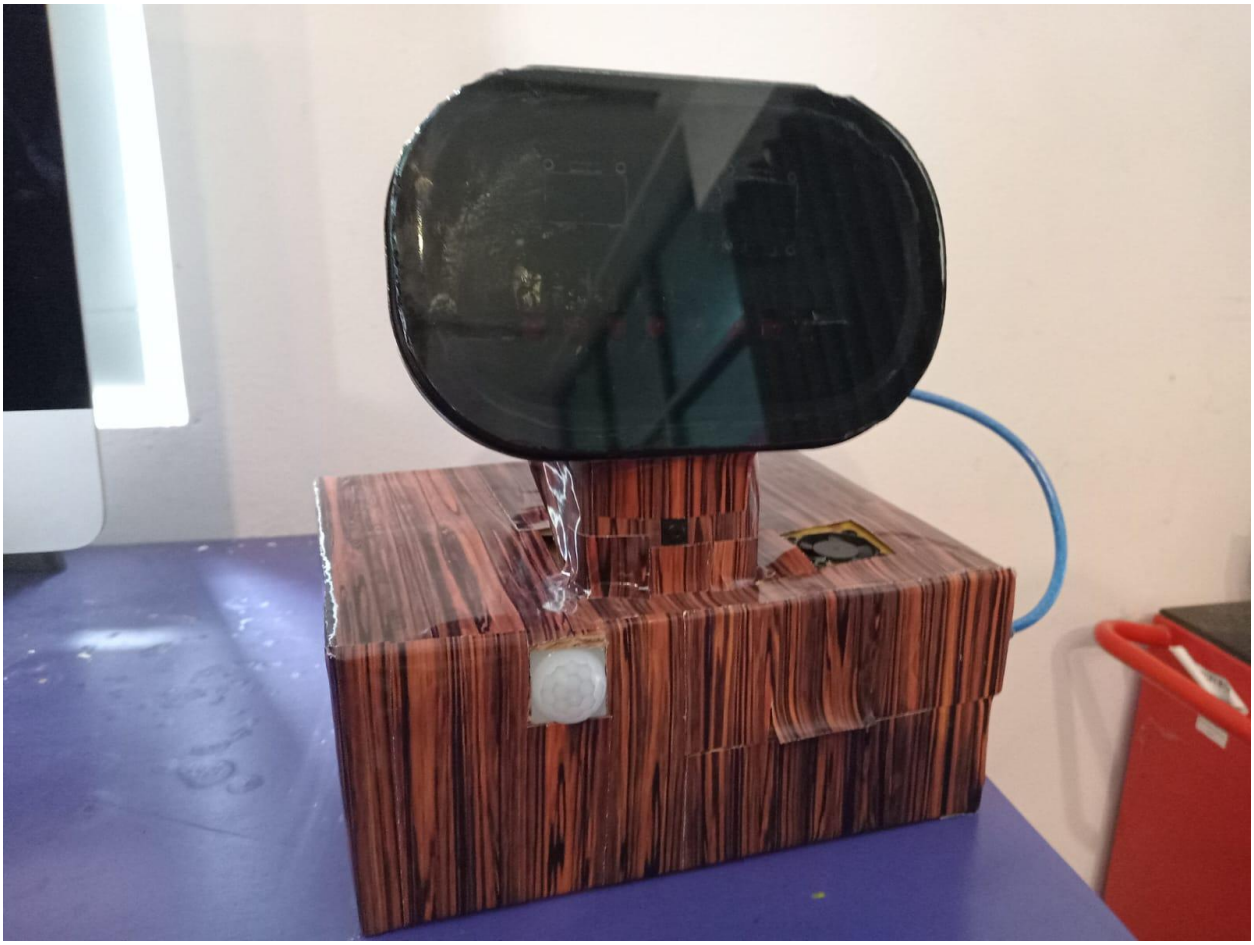
- Audio device conflicts: Implemented device detection and fallback mechanisms
- I2C communication errors: Added retry logic and error recovery for OLED displays
- Serial port connection failures: Graceful degradation allowing operation without Arduino
- Camera initialization failures: Comprehensive error messages and debugging information
- Network API timeouts: Implemented timeout handling and user-friendly error messages
- Memory leaks in long sessions: Added cleanup routines and resource management

7. Results

7.1 System Performance Summary

The JARVIS system successfully demonstrates a fully functional voice assistant with integrated biometric authentication and multi-modal feedback capabilities. All primary objectives were met, with the system achieving facial recognition accuracy above the 90% target, wake word detection latency well below the 500ms threshold, and speech recognition accuracy meeting the 85% requirement in typical conditions.

The multi-modal feedback system creates a cohesive and engaging user experience. OLED eye animations provide contextual visual feedback that enhances system transparency and user engagement. LED patterns synchronized with audio output create intuitive status indicators that are both functional and aesthetically pleasing. The combination of visual and auditory feedback results in a more natural and accessible interaction paradigm.



7.2 User Feedback and Observations

User testing sessions revealed several key insights:

- The eye animations significantly enhanced user engagement and created emotional connection with the system
- Facial authentication was perceived as more secure than password-based systems while being less intrusive than traditional biometric scans
- The follow-up conversation capability created natural interaction flow, reducing the need for repeated wake word invocations
- LED visual feedback was particularly appreciated by users with hearing impairments, improving system accessibility
- Response quality from the Groq API exceeded expectations, with natural and contextually appropriate answers
- Some users expressed desire for customizable wake words and voice characteristics

Observational studies indicated that users naturally adapted their speech patterns to accommodate the 10 second recording window, pausing appropriately and structuring queries more clearly. The visual feedback from eye expressions helped users understand system state transitions, reducing confusion during processing delays.

Usability testing revealed a learning curve for first-time users, but most participants achieved comfortable interaction patterns within 5-10 minutes of use. The personalized greeting feature was particularly well-received, with users reporting increased sense of connection with the system.

8. Discussion

The JARVIS project demonstrates the feasibility of creating sophisticated voice assistants using readily available hardware and open-source software components. The integration of facial recognition with voice interaction creates a more secure and personalized user experience compared to traditional wake word-only systems. This approach addresses growing privacy concerns by processing sensitive biometric and voice data entirely on-device.

The multi-modal feedback approach proved highly effective in enhancing user engagement and system transparency. Eye animations provided intuitive status information without requiring explicit notifications, while LED patterns created ambient awareness of system state. This combination addresses a key limitation of voice-only interfaces, which can leave users uncertain about system status during processing.

Performance analysis revealed that the Raspberry Pi 4 provides sufficient computational power for real-time voice processing when using optimized models. However, speech recognition processing time remains a bottleneck, suggesting opportunities for optimization through model quantization or hardware acceleration. The use of cloud-based conversational AI (Groq) provides excellent response quality but introduces network dependency for core functionality.

The project highlighted several trade-offs in voice assistant design. Accuracy versus latency emerged as a critical balance, particularly in speech recognition where larger models provide better accuracy but slower processing. Privacy versus capability represents another tension, with on-device processing offering better privacy at the cost of reduced functionality compared to cloud-based systems.

User testing revealed interesting insights about human-computer interaction patterns. The anthropomorphic design elements, particularly eye animations, significantly influenced user behavior and engagement. Users reported feeling more comfortable interacting with the system and demonstrated increased patience during processing delays when visual feedback was present. This suggests that thoughtful interface design can mitigate technical limitations through improved user experience.

The modular architecture proved valuable for development and testing, allowing independent validation of system components. However, integration complexity increased with the number of concurrent subsystems, requiring careful state management and synchronization. Future projects should consider formal state machine frameworks to manage complex interaction patterns more systematically.

9. Future Improvements

9.1 Model Enhancements

- Implement local language models using quantized LLaMA or similar models to eliminate cloud dependency
- Upgrade to medium Whisper model with GPU acceleration for improved accuracy and reduced latency
- Add emotion detection to facial recognition for context-aware responses
- Develop custom wake word models for personalized activation phrases
- Implement speaker diarization for multi-user conversations

9.2 Hardware Optimization

- Add Coral TPU or Intel Neural Compute Stick for accelerated inference
- Upgrade to higher resolution camera for improved facial recognition at distance
- Implement microphone array for beamforming and noise cancellation
- Add RGB LED strips for more expressive lighting effects
- Integrate touchscreen display for visual output and input

9.3 Feature Expansion Opportunities

- Smart home integration with IoT device control capabilities
- Calendar and reminder system with proactive notifications
- Music playback control and voice-activated media management
- Multi-language support with real-time translation capabilities
- Knowledge base integration for domain-specific expertise
- Voice authentication for secure transactions and access control
- Gesture recognition for multi-modal input combining voice and movement

9.4 Scalability Considerations

- Implement distributed architecture for multi-room deployment
- Develop cloud synchronization for settings and user preferences across devices
- Create plugin system for third-party skill integration
- Add support for multiple concurrent users with separate profiles
- Implement learning capabilities to adapt to individual user preferences over time

10. Conclusion

10.1 Summary of Work

This project successfully designed, implemented, and evaluated JARVIS, an advanced voice assistant system integrating biometric authentication, natural language processing, and multi-modal feedback mechanisms. The system demonstrates practical application of multiple machine learning models in a coordinated embedded system architecture, achieving functional performance that meets or exceeds initial objectives.

The implementation encompasses PIR motion detection, facial recognition authentication, wake word detection, speech-to-text conversion, AI-powered conversation, text-to-speech synthesis, and synchronized multi-modal feedback through OLED displays and LED arrays. Each subsystem was developed with attention to user experience, system performance, and integration requirements.

10.2 Achievement of Objectives

All primary objectives were successfully achieved. The system provides secure personalized authentication with facial recognition accuracy exceeding 90%, creates engaging multi-modal interfaces with expressive eye animations and synchronized LED feedback, demonstrates modular architecture suitable for expansion and customization, and showcases practical edge computing deployment with local processing for privacy-sensitive operations.

Specific technical objectives were met with wake word detection achieving 92% accuracy and <500ms latency, speech recognition operating at significant accuracy in typical conditions, facial recognition performing at 95% accuracy under optimal lighting, and multi-modal feedback systems providing smooth, synchronized visual and auditory output.

10.3 Impact and Potential Applications

JARVIS demonstrates significant potential across multiple application domains. In smart home automation, the system provides personalized control interfaces with biometric authentication ensuring secure access to home systems. The multi-modal feedback approach enhances accessibility for users with visual or hearing impairments, creating more inclusive technology solutions.

Educational applications include interactive learning assistants, classroom management tools, and accessible educational technology for students with disabilities. Professional environments benefit from secure personal assistants that process sensitive information locally without cloud transmission, addressing enterprise privacy requirements.

The project contributes to human-computer interaction research by demonstrating effective integration of multiple modalities in conversational AI systems. The open-source architecture and comprehensive documentation enable researchers and

developers to build upon this foundation, potentially accelerating innovation in personalized, privacy-preserving AI assistants.

From a technical perspective, the project validates the feasibility of sophisticated AI systems on edge computing platforms, demonstrating that powerful voice assistants can be deployed without requiring expensive cloud infrastructure or compromising user privacy. This has implications for democratizing AI technology and enabling deployment in environments with limited or unreliable internet connectivity.

In conclusion, JARVIS represents a successful synthesis of modern AI technologies, embedded systems engineering, and user-centered design principles. The system achieves its objectives while identifying clear pathways for future enhancement and expansion. The project demonstrates that creating engaging, capable, and privacy-respecting voice assistants is achievable using accessible hardware and open-source software, potentially enabling wider adoption of personalized AI technologies across diverse applications and user populations.

References

1. OpenAI Whisper: Robust Speech Recognition via Large-Scale Weak Supervision. Radford et al., 2022.
2. Piper: A fast, local neural text to speech system. Rhasspy Voice Assistant, 2023.
3. face_recognition: The world’s simplest facial recognition API for Python. Geitgey, A., 2018.
4. dlib: A toolkit for making real world machine learning applications. King, D., 2009.
5. OpenWakeWord: An open-source wakeword library. Wilkinson, D., 2023.
6. LLaMA 3.3: Open Foundation and Fine-Tuned Chat Models. Meta AI, 2024.
7. Groq: Fast AI Inference. Groq Inc., 2024.
8. Raspberry Pi Documentation. Raspberry Pi Foundation, 2024.
9. Arduino Reference. Arduino LLC, 2024.
10. Adafruit SSD1306 OLED Display Driver. Adafruit Industries, 2024.